



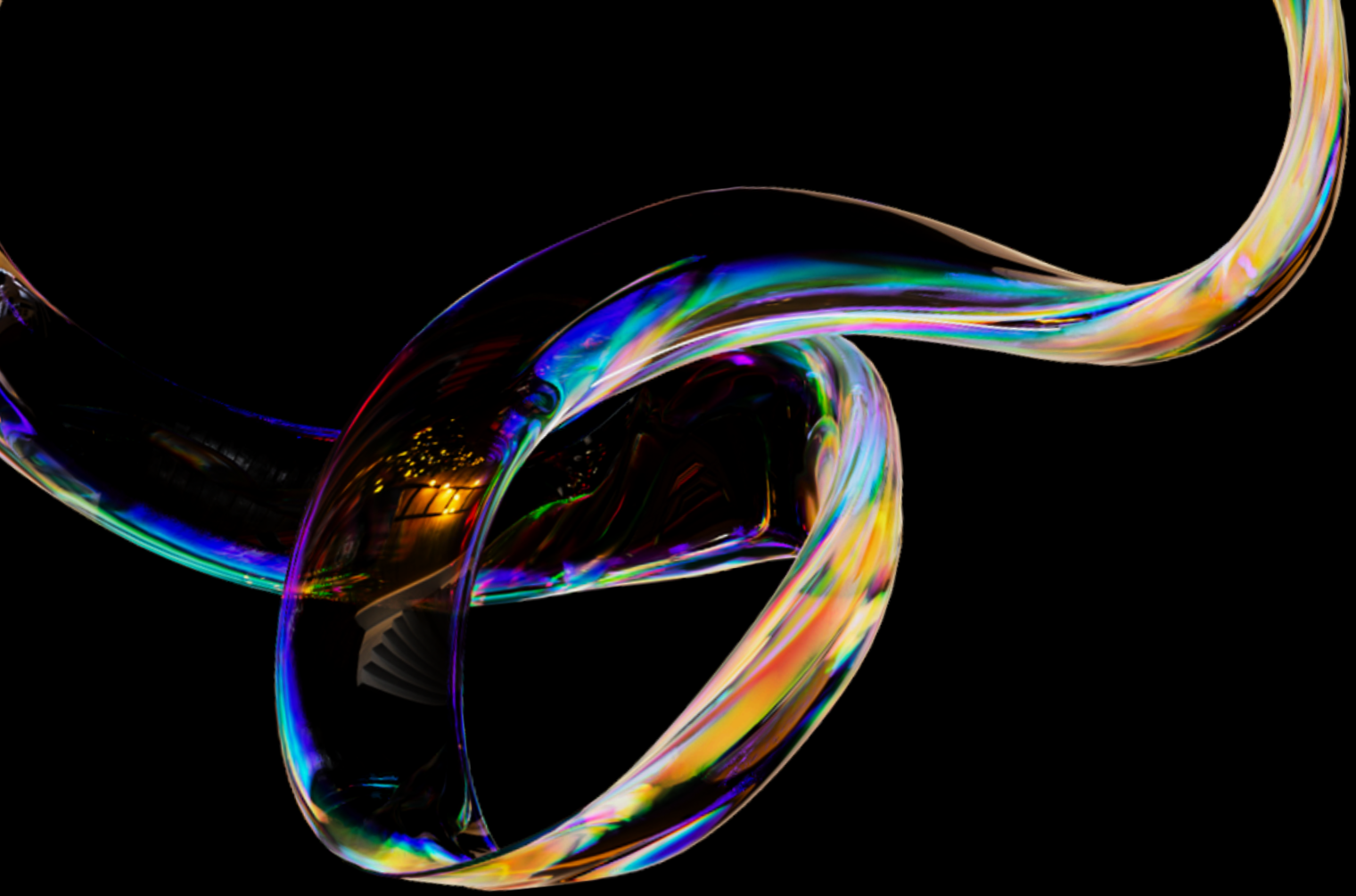
Sheryians coding school

HOW JAVASCRIPT WORKS?

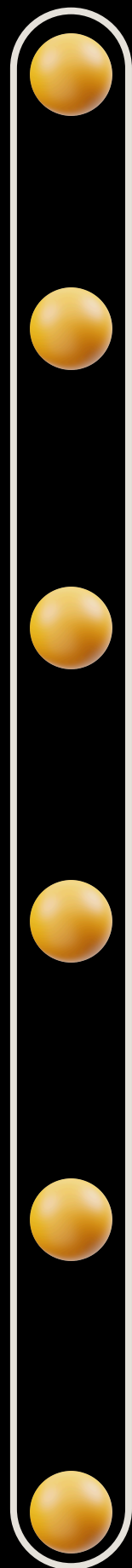
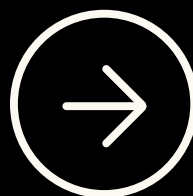
 Present by Carry Saini

 sainicarry@gmail.com

 [sainicarry](#)



JAVASCRIPT RUNTIME ENVIRONMENT



Three Core Truths & The Myth

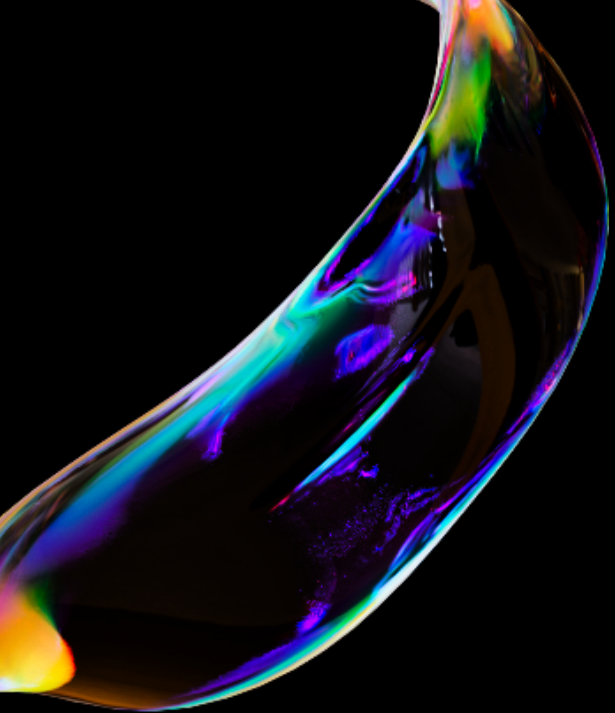
JavaScript Engines

Execution Context & Call Stack Workflow

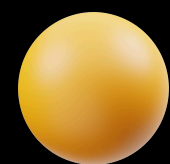
Hoisting Under the Hood

Scope & The Scope Chain

Closures

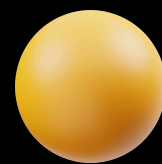


THREE CORE TRUTHS & THE MYTH { }



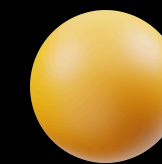
1. JavaScript is Single-Threaded

thread ka mtlb hota hai path, mtlb js ke pass sirf ek hi path hota h apne code ko execute krne ka and iska mtlb yeh h ki js ek time pe ek hi path p run kr skta hai kind of ek time pe sirf ek hi code run kr skta hai which we call as BLOCK CODING.



2. JavaScript is Synchronous

synchronous ka mtlb hota hai ek system mtlb js ka code hmesha ek system ko follow krta hai or code hmesha top to bottom run hota hai line by line by default. and jbtk line 2 ka code execute n ho jae line 3 ka code run nhi hota.



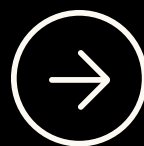
3. JavaScript is Interpreted and JIT-Compiled

in early times log javascript ko ek slow and interpreted language bolte the joki line by line code ko pdhti thi and execute krti thi but now the time has changed. javascript uses JIT compilation to run its code let's discuss it in detail on the next page.



JS IS INTERPRETED AND JIT-COMPILED

- Interpretation: Jab tum page load karte ho, interpreter fatfat code ko bytecode mein bdl kar run kar deta hai taaki user ko wait na karna pade (Fast Startup). and by the side second thing work krti hai i.e.
- Compilation: Jab code chl rha hota hai, toh engine background mai dekhta hai ki kon sa function baar-baar call ho rha hai and these are termed as HOT CODE and Engine us function ko firstly compile kr deta hai and use direct super-fast machine code mai bdl deta hai.



MYTH ABOUT JS

Myth: "Agar JS single-threaded hai, toh setTimeout ka 5 second ka timer chalte waqt bhi hum page par click kaise kar paate hain? JS parallel kaam kar rahi hai, toh multi-threaded hui na?"

Reality: JS engine single-threaded hi hai. wo timer chl hi nhi rha! Jb aap setTimeout likhte ho, toh JS ka engine browser ko bolta hai: ki yeh 5 second ka timer apne pas chlao, mera thread free rakho. and browser ke paas C++ mai likhe hue alag background threads hote hain like Web APIs jo us timer ko handle kr lete hai.



JAVASCRIPT ENGINE

somehow javascript apna code khudse run nhi kr skta usko ek engine ki jrurt hoti h to run it's code and alag alag platform ke alag alag engines hote hai jisko wo use krte hai.

V8	Chrome, Node.js, Edge (Google)
SpiderMonkey	Firefox (Mozilla)
JavaScriptCore	Safari (Apple)

TOKENIZING / LEXING

Engine code ko ek bdi string ki trh dekhta hai and jha jha use tokens milte hai wha wha wo code ko break kr deta hai.

IGNITION (BYTECODE GENERATION)

engine AST ko bytecode mai convert kr deta hai. it is intermediate low-level code jo engine ke andr chlta hai.and memory bohot kam hoti h

PARSING —> AST (TREE BANANA)

jo tokens seperate kiye hai aab unko ek tree ki form me create kiya jata hai which we termed as AST (Abstract Syntax Tree)

TURBOFAN (OPTIMIZATION)

agr koi loop jyada bda hota hai toh engine usko binary language m convert kr deta hai jis se interpreter pe oad nhi pdta and CPU se loop run ho jata hai

Agar tumne syntax mein galti ki (jaise let x = ;), toh parser AST nahi bana payega aur code execution se pehle hi crash ho jayega (SyntaxError)



EXECUTION CONTEXT

now come the most important and the most interesting part of javascript, i.e. execution context. it is a box like structure jo bnta hai jab hm kisi code ko run krte hai. there are 2 main types of execution context

GLOBAL EXECUTION CONTEXT

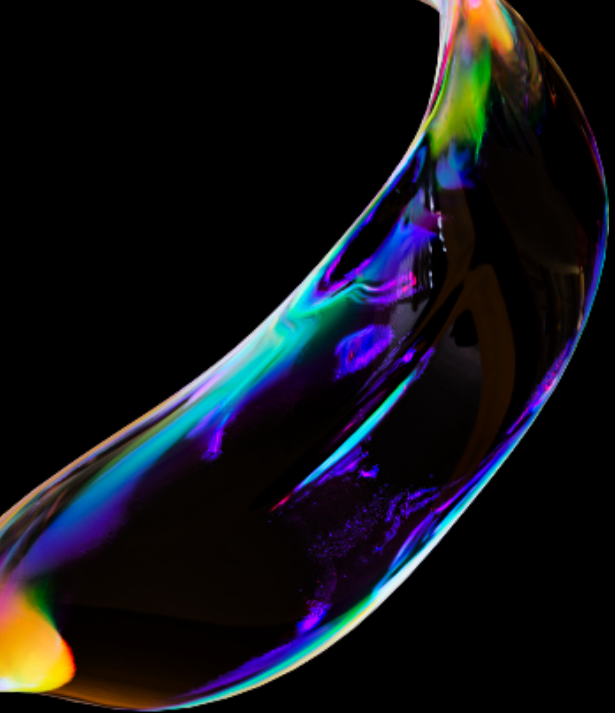
- WHENEVER a function is runned , ek default box create hota hai runtime mai jiske andr sara code 2 phase mai tut ke likha jata hai.
- Browser mai yeh user ke liye window object bnata h or this ko window ki traf point krwata hai.

FUNCTION EXECUTION CONTEXT

- jab bhi ek function ko call kiya jata hai, define nhi call kiya jata hai toh us position pe global execution context mai ek private inner box create ho jata hai jisme function ke andr ka code rkha jata hai.
- Jab function khatam hota hai, toh uska FEC box delete ho jata hai.

let's discuss in detail what happens inside an execution context.





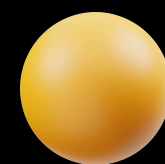
THE GOLDEN CORE: THE TWO PHASES OF EXECUTION CONTEXT



Creation Phase (Memory Allocation)

is phase mai code run nhi hota blki engine pure code ko scan krta hai or jitne bhi variables and functions create hote hai uski memory allocate krta hai.

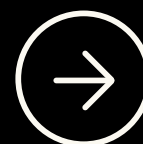
like “var a = 12;” toh creation phase is line ko 2 phase mai tod deta hai `var a;` and `a = 12` and jo variable declaration wali line hai wo sbse upr chli jati hai and uski memory alocate ho jati hai



Execution Phase (Code Running)

aab jo hmara a=12 hua tha mtlb variable initialization is phase mai hoti hai jitne bhi variables ko koi value assign hoti hai wo is phase mai hoti hai and jitne bhi function call hote hai wobhi ishi phase mai hote hai.

jab function call hota hai to execution phase mai hi uska ek function inner execution context bnke ready ho jata hai and jaiye global execution mai 2 phase hote hai usme bhi same yehi process hoti hai run krne ki.



EXAMPLE :

- Phase 1 (Creation): Engine ne dekha var x, memory mai dala x= undefined phir dekha function greet, memory mein pura structure dal diya greet: {...code...}.
- Phase 2 (Execution): Line 1 par aaya, ab x ki value undefined se badal kar 10 ho gayi. Line 2-4 ko skip kiya (kyuki wo sirf definition hai). Line 5 par aaya: greet(). Function call hua! at that time greet function ke liye ek naya private Function Execution Context bana aur uske andar ka code chala joki tha console.log("hello");

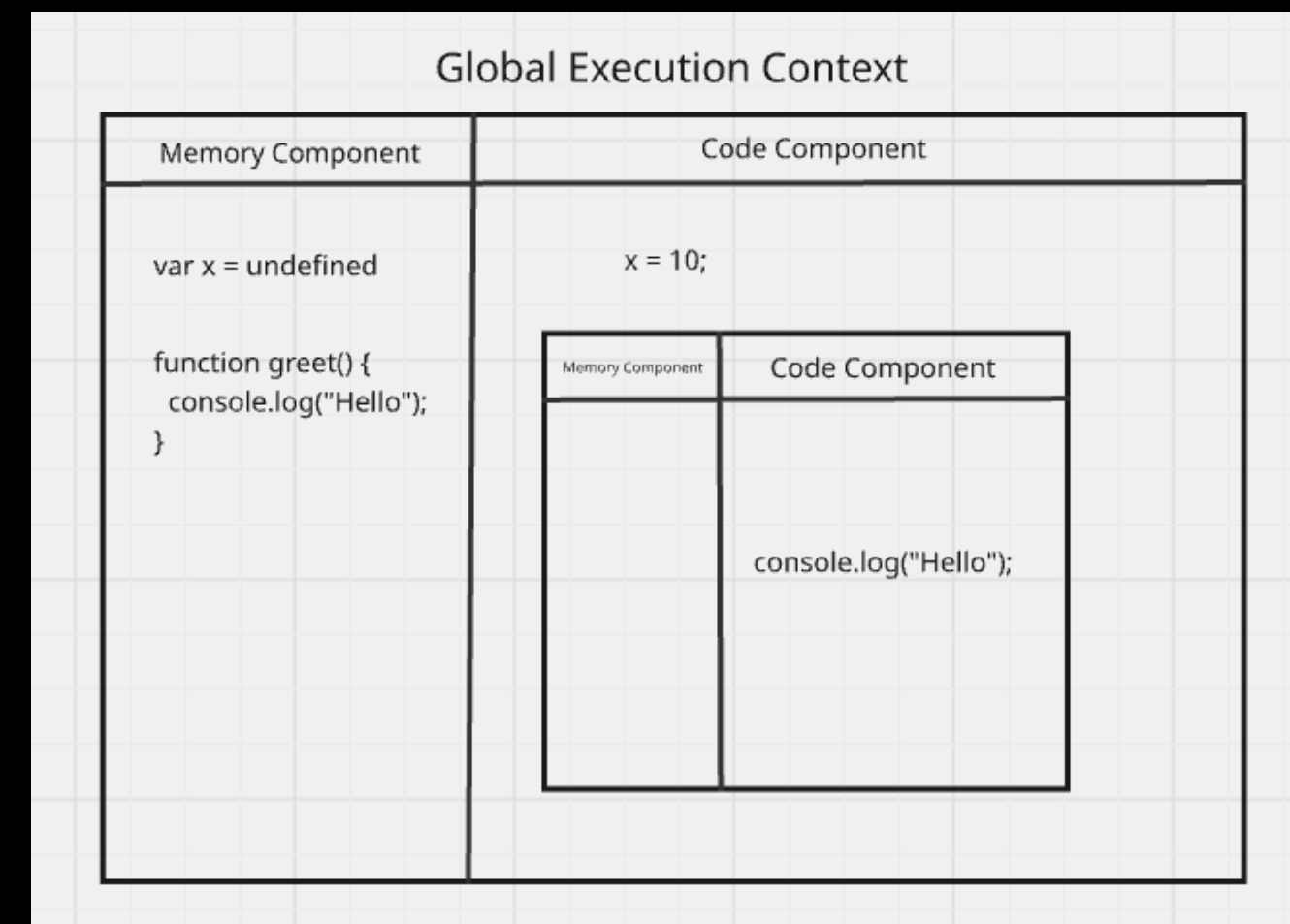
agar koi function khudke andr hi khudko baar baar bina ruke call karta hai toh we call it as stack overflow(CRASH).

```
function loop() {  
  loop();  
}  
loop();
```

RangeError: Maximum call stack size exceeded



```
1  var x = 10;  
2  function greet() {  
3    console.log("Hello");  
4  }  
5  greet();
```

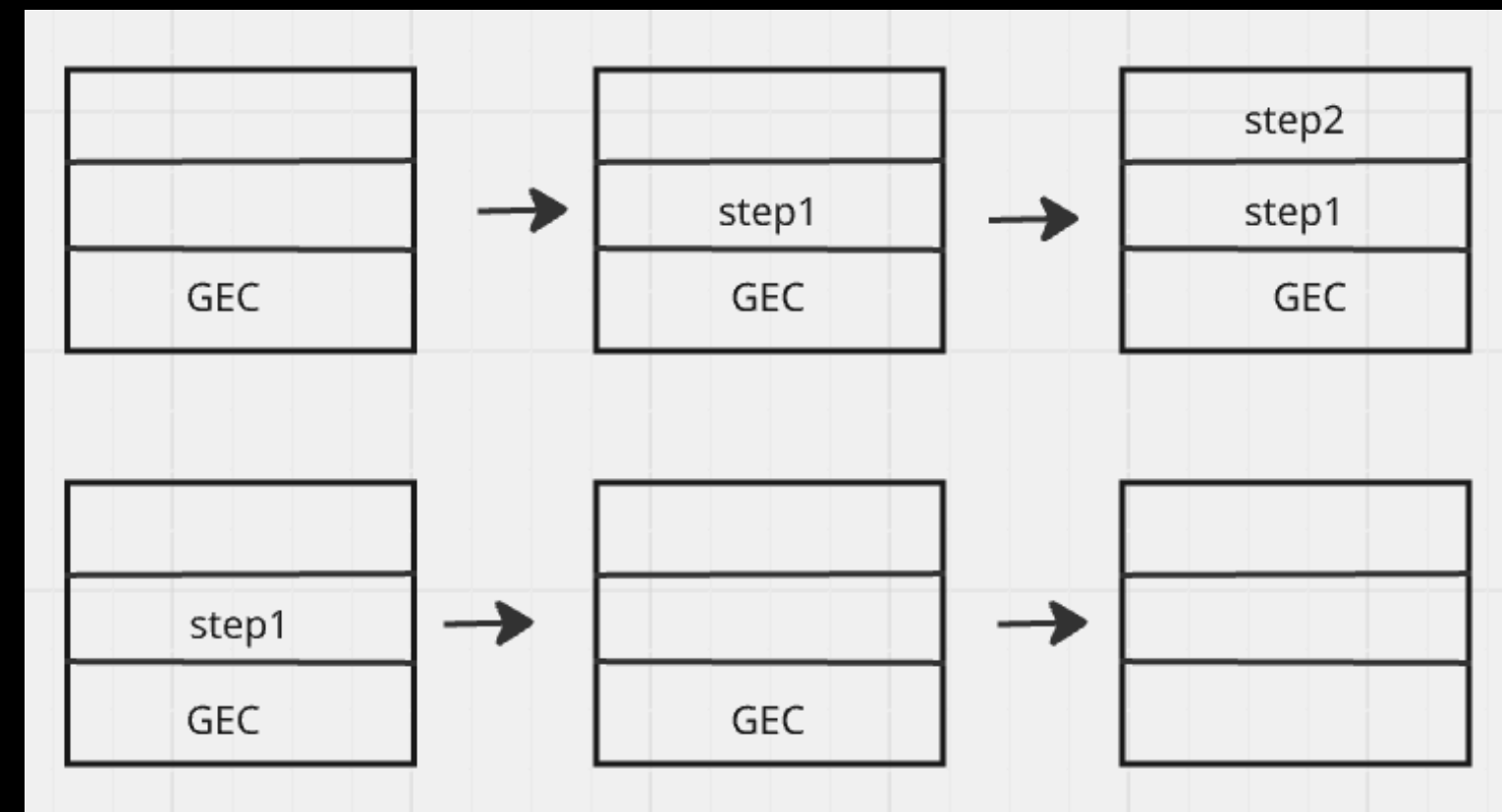


CALL STACK

call stack javascript engine ka ek built in tracking system hota hai jo yaad rkhta hai ki abhi kon sa function execute ho rha hai or us function ke baad kis line par waps aana hai. yeh LIFO pai work krta hai which stands for Last in First Out. mtlb jo function sbse end mai call hua hai wohi sbse phle khatam hoga or stack se bahr niklega.



```
1  function step2() {  
2      console.log("Finishing...");  
3  }  
4  
5  function step1() {  
6      step2();  
7  }  
8  
9  step1();
```



HOISTING

jab hm kisi function yah variable ko declare krke initialize krne se phle hi access krne ki kosis krte hai usko hoisting khte hai let var and const ke liye yeh alag alag trh se work krte hai. let's discuss

Var

jab bhi hm kisi variable ko var se create krte hai or usko initialize krne se phle access krte hai to hme return mai undefined milta hai due to execution context memory allocation.

```
1 console.log(a);  
2 var a = 5;
```

let and const

is case m bhi hoisting hoti hai but temporal zone ke sath. mtlb scope ke suru hone se variable declaration tk bich m jitna code hota hai wo TDZ bn jata hai or usme agr hm variable ko access krenge to reference error milega .

Function Declarations vs Function Expressions

agr function declare krte ho toh uski body hoist ho jati hai or agr tum khu bhi use excess krne ki kosis krte ho toh function call ho jata hai.

jab function ko variable mai store krte ho toh usme variable ke rules lg jate hai hoisting ke.

```
1 sayHi(); // "Hi"  
2 function sayHi() { console.log("Hi"); }  
3  
4 sayBye(); // reference error  
5 var sayBye = function() {  
6     console.log("Bye"); }  
7
```

SCOPE

scope ka sidha mtlb hai aapka variable code mai kin boundaries tk valid rhega aap unko kha kha access kr paoge

Global Scope

jo variable sbse bahr declare hota hai or usko pure code mai kahi bhi access kr skte hai unhe gobal scope variable khte hai.

Function Scope

agr koi varibale kisi function ke andr bne hai to wo sirf us function mai hi access ho paynge.
var, let, const teeno function scoped hai or function ke andr bnaye jate hai

Block scope

block mtlb hota hai {} curly braces ke andr ka area
let and const block scope hote hai and var block ki restriction ko follow nhi krta.

Lexical Scope

lexical word ka tmlb hota hai physical location jha code likha gya ho. javascript mai child function apne parent mai declare hue variables ko bhi access kr skta hai.

```
1 let name = "Global";
2 function outer() {
3   let name = "Lexical";
4   function inner() {
5     console.log(name);
6     inner();
7   }
8   outer();
```




THE SCOPE CHAIN

jab aap koi code likhte hai to engine us variable ko kaise dhundta hai, woh phle us local scope mai dhundta hai agr nhi milta tohh firr parent p move krta hai or firr uske parent pe.

- When inner() logs c: Found immediately in its own scope (\$30\$).
- When inner() logs b: Not found locally —> Goes up to outer() scope —> Found (\$20\$).
- When inner() logs a: Not found locally —> Not found in outer() —> Goes up to Global scope —> Found (\$10\$)



CLOSURES

Jab ek function kisi dusre function ke andar se return hota hai, toh wo sirf function code return nahi karta, balki apne Lexical Environment ke saare variables ka access apne sath baandh kar (bundle karke) le jata hai. Is bundle ko Closure kehte hain.

```
1  function counter() {  
2    let count = 0;  
3    return function () {  
4      count++;  
5      return count;  
6    };  
7  }  
8  
9  const inc = counter();  
10 console.log(inc()); // 1  
11 console.log(inc()); // 2
```

counter() call hua. Stack mein iska execution context bana, jahan count = 0 hai.

counter() ne inner function return kiya aur stack se pop off (delete) ho gaya. Normal rule ke mutabik count variable ko memory se delete ho jana chahiye tha.

Lekin V8 engine ne dekha ki jo function return hua hai, wo count ko use kar raha hai. Engine ne count ko garbage collect nahi kiya, balki use Heap Memory mein zinda rakha.

Ab jab tum inc() ko call karte ho, toh wo har baar usi purane saved count variable ko access karta hai aur use increment karta hai (1, 2, 3)



CONCLUSION

javascript mai jab bhi hm koi action lete hai yah koi code likhte hai to usko “magic” samajhte hai but wooh actually engine ka solid design hota hai code kbhi bhi direct nhi chlta uske liye sbse phle execution context ready hota hai and hoisting scope or closurs sath mai work krte hai and call stack (LIFO) hmare pure code ke flow ko control kra hai.

yhi farq ek normal coder ko ek pro software developer banata hai, jo aage jaakar bina kisi bug ke solid aur fast web applications design kar sake.